

CLASS 07 — TRANSLATING THE LEVERAGE - NETWORK APPLICATION

Stata, R, Python

STEP 1 • FILTER THE SAMPLE

Loading .dta and filtering rows

Watch missing-value semantics: in Stata, missing (.) is treated as $+\infty$ in comparisons, so `drop if at < 0` silently keeps missing at — R/Python do not behave the same way.

STATA	R	PYTHON
<pre>use "data1.dta", clear keep if fic == "USA" drop if missing(at) drop if at < 0 destring sic, replace drop if sic >= 6000 /// & sic <= 6999 count</pre>	<pre>library(haven); library(dplyr) library(tidyr) data <- read_dta("data1.dta") data <- data %>% filter(fic == "USA") %>% drop_na(at) %>% filter(at >= 0) %>% mutate(sic = as.numeric(sic)) %>% filter(!(sic >= 6000 & sic <= 6999)) nrow(data)</pre>	<pre>import pandas as pd data = pd.read_stata("data1.dta") data = data[data["fic"]=="USA"].copy() data = data.dropna(subset=["at"]) data = data[data["at"] >= 0] data["sic"] = pd.to_numeric(data["sic"], errors="coerce") data = data[~((data["sic"] >= 6000) & (data["sic"] <= 6999))] len(data)</pre>

⚠: Stata's `drop if at < 0` keeps missings (`. > 0` is `TRUE`). In R/Python, `NA/NaN` comparisons return `NA/False` — explicit `!is.na()` / `.notna()` matters.

STEPS 2 & 3 · BUILD VARIABLES & HANDLE OUTLIERS

gen / egen → create variables; winsor packages diverge

Variable construction is trivial in all three languages — the friction is the winsorization. Stata's `winsor x, gen(Wx) p(0.01)` is one line; in R and Python is more complex. To match Stata exactly, follow the code below. Always sanity-check min/max before vs after.

STATA

```
gen lev = (dltt+dlc)/at
gen mb = (csho*prcc_f
          + at - ceq)/at
gen roa = ebitda/at
gen sic2 = int(sic/100)

winsor lev, gen(Wlev) p(0.01)
winsor mb, gen(Wmb) p(0.01)
winsor roa, gen(Wroa) p(0.01)
winsor fcf, gen(Wfcf) p(0.01)
```

R

```
# Stata-exact winsor (no interp)
wins <- function(x, p = 0.01) {
  s <- sort(x[!is.na(x)])
  N <- length(s)
  p1 <- as.integer(p * N)
  lo <- s[p1+1]; hi <- s[N-p1]
  out <- x
  out[!is.na(x) & x < lo] <- lo
  out[!is.na(x) & x > hi] <- hi
  out }

data <- data %>% mutate(
  lev = (dltt+dlc)/at,
  mb = (csho*prcc_f+at-ceq)/at,
  roa = ebitda/at,
  sic2 = floor(sic/100),

  Wlev = wins(lev,0.01),
  Wmb = wins(mb, 0.01),
  Wroa = wins(roa,0.01),
  Wfcf = wins(fcf,0.01))
```

PYTHON

```
data["lev"] = (data["dltt"]
               + data["dlc"]) / data["at"]
data["mb"] = (data["csho"]*data["prcc_f"]
              + data["at"] - data["ceq"])/data["at"]
data["roa"] = data["ebitda"]/data["at"]
data["sic2"] = (data["sic"]//100
               ).astype("Int64")

def wins(x, p=0.01): # Stata-exact
    arr = np.asarray(x, float)
    s = np.sort(arr[~np.isnan(arr)])
    N = len(s); p1 = int(p*N)
    lo, hi = s[p1], s[N-p1-1]
    out = arr.copy(); m = ~np.isnan(arr)
    out[m] = np.clip(arr[m], lo, hi)
    return out

data["Wlev"] = wins(data["lev"],0.01)
data["Wmb"] = wins(data["mb"], 0.01)
data["Wroa"] = wins(data["roa"],0.01)
data["Wfcf"] = wins(data["fcf"],0.01)
```

STEP 4 • DESCRIBE & UNIVARIATE TEST

tabstat / corr / ttest → R & Python equivalents

After filtering and winsorizing, pick a clean analysis sample — only rows where every variable you'll use in the regression is non-missing. Then on that sample you produce two basic outputs: descriptive statistics (means, standard deviations, percentiles) and a correlation matrix. Finally, run a univariate t-test to ask a simple question.

STATA	R	PYTHON
<pre>egen miss = rowmiss(network Wlev /// ln_age ln_nseg Wmb Wfcf Wroa) gen sample = (miss == 0) drop miss tabstat network Wlev age nseg /// Wmb Wfcf Wroa if sample, /// stats(mean sd p10 p50 p90 count) corr network Wlev age nseg /// Wmb Wfcf Wroa if sample ttest network = 0 if sample egen mean_lev = mean(Wlev) if sample gen hi_lev = (Wlev > mean_lev) if sample ttest network if sample, by(hi_lev) ttest network if sample, by(hi_lev) unequal</pre>	<pre>reg_vars <- c("network","Wlev", "ln_age","ln_nseg","Wmb", "Wfcf","Wroa") data\$sample <- complete.cases(data[, reg_vars]) sample_data <- data %>% filter(sample) desc_vars <- c("network","Wlev","age", "nseg","Wmb","Wfcf","Wroa") psych::describe(sample_data[,desc_vars], quant = c(.10, .50, .90)) cor(sample_data[, desc_vars], use = "complete.obs") t.test(sample_data\$network, mu = 0) mean_lev <- mean(sample_data\$Wlev, na.rm = TRUE) sample_data\$hi_lev <- sample_data\$Wlev > mean_lev t.test(network ~ hi_lev, data=sample_data, var.equal=TRUE) t.test(network ~ hi_lev, data=sample_data, var.equal=FALSE)</pre>	<pre>reg_vars = ["network","Wlev", "ln_age","ln_nseg","Wmb", "Wfcf","Wroa"] data["sample"] = data[reg_vars] \ .notna().all(axis=1) sample = data[data["sample"]].copy() desc_vars = ["network","Wlev","age", "nseg","Wmb","Wfcf","Wroa"] sample[desc_vars].describe(percentiles=[.10,.50,.90]).T sample[desc_vars].corr() stats.ttest_1samp(sample["network"].dropna(), 0) mean_lev = sample["Wlev"].mean() sample["hi_lev"] = sample["Wlev"] > mean_lev g0 = sample.loc[~sample.hi_lev,"network"].dropna() g1 = sample.loc[sample.hi_lev,"network"].dropna() stats.ttest_ind(g0,g1, equal_var=True) stats.ttest_ind(g0,g1, equal_var=False)</pre>

STEP 5 • ESTIMATE WITH FIXED EFFECTS

i.fyear i.sic2 and panel regressions

Stata bakes factor variables (i.fyear, i.sic2) and panels (xtset) into the syntax. In R, declare the panel via `plm::pdata.frame()`; pooled OLS and +controls/FE columns are `lm()` with `factor(fyear) + factor(sic2)`, and firm FE is `plm(..., model = 'within')`. In Python, set a MultiIndex on (firmid, fyear), use `smf.ols` with `C(fyear) + C(sic2)` for the OLS columns, and `PanelOLS` with `C(fyear) + EntityEffects` for the firm-FE column.

STATA

```
xtset firmid fyear

* (1) pooled OLS
reg Wlev network if sample

* (2) + controls + year + ind FE
reg Wlev network ln_age ln_nseg ///
    Wmb Wfcf Wroa i.fyear i.sic2 ///
    if sample

* (4) firm FE + year FE
xtreg Wlev network ln_age ln_nseg ///
    Wmb Wfcf Wroa i.fyear ///
    if sample, fe vce(cluster firmid)
```

R

```
library(plm)
pdata <- pdata.frame(sample_data,
  index = c("firmid", "fyear"))

# (1) pooled OLS
reg1 <- lm(Wlev ~ network,
  data = sample_data)

# (2) + controls + year + ind FE
reg2 <- lm(Wlev ~ network + ln_age
  + ln_nseg + Wmb + Wfcf + Wroa
  + factor(fyear) + factor(sic2),
  data = sample_data)

# (4) firm FE + year FE
reg4_fe <- plm(Wlev ~ network + ln_age
  + ln_nseg + Wmb + Wfcf + Wroa
  + factor(fyear),
  data = pdata, model = "within")
```

PYTHON

```
from linearmodels.panel import PanelOLS
panel = sample.set_index(
  ["firmid", "fyear"])

# (1) pooled OLS
reg1 = smf.ols("Wlev ~ network",
  data=sample).fit()

# (2) + controls + year + ind FE
reg2 = smf.ols(
  "Wlev ~ network + ln_age + ln_nseg"
  " + Wmb + Wfcf + Wroa"
  " + C(fyear) + C(sic2)",
  data=sample).fit()

# (4) firm FE + year FE
reg4 = PanelOLS.from_formula(
  "Wlev ~ network + ln_age + ln_nseg"
  " + Wmb + Wfcf + Wroa"
  " + C(fyear) + EntityEffects",
  data=panel).fit(
  cov_type="clustered",
  cluster_entity=True)
```

STEP 5 • SES, TESTS & OUTPUT

vce(cluster ...), test, outreg2 — and how to get them elsewhere

Stata attaches clustered SEs and joint tests directly to the estimator. In R you layer them on: refit-then-test with sandwich (vcovCL for lm; vcovHC type='sss' for plm to match Stata's small-sample correction), coeftest for coefficients, car::lht for joint tests — passing the SAME vcov each time. In Python, statsmodels takes cov_type='cluster' directly on .fit(), and PanelOLS takes cov_type='clustered', cluster_entity=True. For tables: outreg2 → stargazer (R) → summary_col (Python).

STATA	R	PYTHON
<pre>* (3) clustered SE at firm level reg Wlev network ln_age ln_nseg /// Wmb Wfcf Wroa i.fyear i.sic2 /// if sample, vce(cluster firmid) * joint tests test network = 0 test Wfcf + Wroa = 0 * output to Excel outreg2 using panel_2026, /// append tstat bdec(3) excel</pre>	<pre># (3) clustered SE on reg2 v_cl <- vcovCL(reg2, type="HC1", cluster = ~ firmid) reg3 <- coeftest(reg2, vcov = v_cl) # joint tests (pass SAME vcov) lht(reg2, "network = 0", vcov=v_cl) lht(reg2, "Wfcf + Wroa = 0", vcov=v_cl) # (4) firm-FE SE: Stata-equivalent v_fe <- vcovHC(reg4_fe, type="sss", cluster = "group") reg4 <- coeftest(reg4_fe, vcov = v_fe) # stargazer regression table stargazer(reg1, reg2, reg2, reg4_fe, report = "vc*t", type = "html", out = "panel_2026x.html")</pre>	<pre># (3) cluster SE at fit time (smf) reg3 = smf.ols("Wlev ~ network + ln_age + ln_nseg" " + Wmb + Wfcf + Wroa" " + C(fyear) + C(sic2)", data=sample).fit(cov_type="cluster", cov_kws={"groups":sample["firmid"]}) # joint tests on reg3 reg3.t_test("network = 0") reg3.t_test("Wfcf + Wroa = 0") # (4) PanelOLS clustered by entity # (see slide 5 for fit call) # HTML output via summary_col table = summary_col([reg1, reg2, reg3], stars=True) open("panel_2026.html", "w").write(table.as_html() + reg4.summary.as_html())</pre>